

Week 7 Report — Cost Optimization & Feedback Loop

Hanfu Zhao · AIPI 561 · 2026-06-22

What I built

Three classes in `cost_optimization_starter.py`:

1. **CostAnalyzer** — logs each query's cost components and flags outliers
2. **OptimizationStrategy** — caching, model routing, retrieval trimming, response compression
3. **FeedbackLoop** — collects corrections gated by role authority, then computes metrics

Plus two scripts:

- `scripts/test_queries.json` — 12 diverse test queries
- `scripts/evaluate_agent.py` — runs queries through the optimized agent and prints cost breakdown, spikes, optimization impact, and feedback metrics

Code layout



Key choices

Spike detection uses population stdev (we have the full population, not a sample). If all costs are identical, stdev is 0 and nothing gets flagged — that's intentional.

Model routing checks two signals: complexity keywords (`analyze`, `compare`, `design`, etc.) and word count > 15. Either one bumps the query to `gemini-2.5-pro`. Simple stuff goes to `gemini-1.5-flash`.

Authority is a flat dict mapping role → level (intern=0 ... executive=4). Submissions need level ≥ 2 (manager+). The correction also has to be longer than the original — a one-word "fix" doesn't count.

Error patterns just pulls keywords out of the original queries (filters stop words, min 4 chars) and counts them. Crude but gives a sense of what topics keep coming back.

Test output (unit tests)

Evaluation script output

Numbers from the run

Unit tests:

- 16 queries tracked, total cost \$2.525
- Spike: \$2.00 query vs threshold \$1.11
- 21% estimated savings from optimizations
- 3 corrections collected, 67% validation rate

Evaluation script (12 queries):

- Total cost \$0.484, avg \$0.040/query
- 50% savings (retrieval + model routing)
- 144 docs saved by retrieval trimming
- 8 queries downgraded to flash model
- 2 corrections, 100% validation rate

How to run

```
bash
```

Only uses stdlib (`json` , `logging` , `math` , `typing` , `datetime`).

Screenshot: Test Output

```
$ python3 cost_optimization_starter.py
```

```
=====
CostAnalyzer Tests
=====
```

```
[PASS] empty breakdown count=0
[PASS] empty breakdown total=0
[PASS] empty spikes
[PASS] 1 record -> count=1
[PASS] 1 record -> no spikes yet
[PASS] 15 queries recorded
[PASS] retrieval total ok
[PASS] llm total ok
[PASS] tool total ok
[PASS] total ok
[PASS] spike detected
[PASS] spike is the expensive one
[PASS] spike has breakdown
```

```
spike: cost=2.0, threshold=1.109114, excess=0.890886
```

```
[PASS] identical costs -> no spikes
```

```
breakdown: {'retrieval_total': 0.65, 'llm_total': 1.3, 'tool_total': 0.375, 'error_total': 0.2, 'total_daily':
2.525, 'query_count': 16}
```

```
=====
OptimizationStrategy Tests
=====
```

```
[PASS] first call miss
[PASS] returns provided response
[PASS] second call hit
[PASS] returns cached
[PASS] different query miss
[PASS] 15->3
[PASS] 10->2
[PASS] 1->1
[PASS] 0->1
[PASS] simple -> flash
[PASS] short -> flash
[PASS] analyze -> pro
[PASS] compare -> pro
[PASS] long -> pro
[PASS] short unchanged
[PASS] long compressed
[PASS] keeps first 5
[PASS] drops later
[PASS] savings > 0
[PASS] strategies listed
[PASS] breakdown dict
[PASS] details dict
```

```
impact: 21.0% savings
```

```
strategies: ['model_selection', 'response_compression', 'caching', 'retrieval_optimization']
```

```
details: {'cache_hits': 1, 'cache_misses': 2, 'docs_saved': 20, 'model_downgrades': 2, 'chars_saved': 82}
```

```
=====
FeedbackLoop Tests
=====
```

```
[PASS] empty total=0
[PASS] empty rate=0
[PASS] manager accepted
[PASS] 1 stored
[PASS] intern rejected
[PASS] still 1
[PASS] engineer rejected
[PASS] shorter rejected
[PASS] unknown role rejected
[PASS] director accepted
[PASS] executive accepted
[PASS] validate idx 0
[PASS] validate idx 1
[PASS] validate -1
[PASS] validate 99
[PASS] correction 0 validated
[PASS] correction 1 validated
[PASS] total=3
```

```
[PASS] rate > 0
[PASS] avg len > 0
[PASS] patterns list

metrics: {'total_corrections': 3, 'validation_rate': 0.67, 'avg_correction_length': 129.0, 'top_error_patterns':
[{'pattern': 'over', 'count': 2}, {'pattern': 'travel', 'count': 1}, {'pattern': 'policy', 'count': 1}, {'pattern':
'flights', 'count': 1}, {'pattern': 'hours', 'count': 1}]}

=====
Integration Test
=====

[PASS] 5 queries in integration
[PASS] integration correction accepted

queries: 5, cost: $0.1650
cache hits: 1, savings: 36.0%
corrections: 1

=====
Results: 59/59 passed, 0 failed
All tests passed!
=====
```

Figure 1: Output of running `python3 cost_optimization_starter.py` — 59/59 tests passed.